Grid World Dual Search Visualizer

A Comparative Study of Classical Pathfinding Algorithms

Swapnanil Bala
M.S. Data Science | Khoury College of Computer Sciences
Northeastern University, Boston MA

Technologies: Python • Pygame • Matplotlib
https://github.com/SwapnanilBala/GridGame_Dual_Search_Visualizer

Abstract

This report presents the design, implementation, and empirical evaluation of the Grid World Dual Search Visualizer — an interactive path-finding toolkit that enables side-by-side comparison of six classical search algorithms on maze-like 2D grid environments. The system implements A*, Breadth-First Search (BFS), Depth-First Search (DFS), Depth-Limited Search (DLS), Uniform-Cost Search (UCS), and Bidirectional Search (BDS), each visualized in real time using Pygame with synchronized dual-pane playback.

The primary inspiration is didactic: visualizing and quantifying algorithmic behavior across varied grid topologies that reveal trade-offs that are (a lot of the time) difficult to communicate through pseudocode alone. Experimental results across five custom maps confirm expected theoretical properties — A* consistently expands the fewest nodes on uniform terrain, UCS finds cost-optimal paths on weighted maps, and BDS achieves roughly 29% node reduction versus single-direction BFS. DLS demonstrates its specialized advantage over DFS on maps with large dead-end corridors, cutting wasted expansions by 25%.

This project is extensible: new algorithms, map formats (There are gen_maps.py/ gen_hard_maps.py files which were used to create the maps), and terrain types can be added with minimal changes to the shared base interface.

1. Introduction

Pathfinding on segmented 2D grids is one of the foundational problems in artificial intelligence, with applications ranging from robotics and game AI to logistics and network routing. While the theoretical properties of classical search algorithms are well-established, their practical behavior - in terms of node expansions, frontier growth, path quality, and sensitivity to terrain - is best understood empirically through direct comparison.

The GridWorld Dual Search Visualizer addresses this gap by providing an interactive environment in which two algorithms can be run concurrently (side-by-side) on the same map, with real-time animation of node expansion and path tracing, followed by automated generation of comparative bar charts and summary statistics. The tool is designed both as a learning aid and as a lightweight experimentation platform.

1.1 Objectives

- Implement six canonical search algorithms with a shared, consistent interface.
- Design terrain-aware grid environments that stress-test algorithm trade-offs.
- Provide real-time visual feedback of search behavior via Pygame.
- Generate quantitative comparison metrics: path cost, path length, nodes expanded, peak frontier size.
- Demonstrate the practical significance of informed vs. uninformed search and cost-blind vs. cost-aware strategies.

2. Background & Related Work

The algorithms implemented in this project are well-documented in the Artificial Intelligence literature. Russell & Norvig (2021) provide the canonical treatment of BFS, DFS, UCS, and A* within the unified framework of graph search agents. The A* algorithm, introduced by Hart, Nilsson & Raphael (1968), remains the benchmark for heuristic-guided pathfinding due to its optimality guarantee under admissible heuristics. Pohl (1971) formalized bidirectional search, demonstrating that searching simultaneously from start and goal can reduce the explored search space from O(b^d) to approximately O(b^(d/2)).

More recent work has extended these ideas to weighted grids and dynamic environments (e.g., D* Lite, Jump Point Search), but the classical algorithms remain the standard baseline for introductory AI systems. Existing visualization tools such as PathFinding.js provide browser-based demos but lack support for simultaneous dual-algorithm comparison and terrain-weighted cost modeling — gaps this project addresses.

With the pygame module the scope of visualization of various grid games have expanded tremendously, prior to this I had worked on building a system where the goal of an agent was to fill every cell of a board (full of empty grids with no colors) with different colors, while ensuring that no colors on any two adjacent grids were not the same, I have always loved the various search algorithms and they had always fascinated me and since with the help of pygame the visualization was possible I decided to proceed with it.

3. System Design & Architecture

The system is organized into four main modules: algorithm implementations, the grid environment, visualization, and utilities. All algorithms implement a shared SearchResult dataclass interface, ensuring that metrics are collected consistently regardless of the underlying search strategy.

3.1 Module Overview

Module	File(s)	Responsibility
Algorithms	algos/search/*.py	A*, BFS, DFS, DLS, UCS, BDS implementations
Environment	env/gridworld.py	Map parsing, neighbor generation, terrain cost lookup
Visualization	visualization/pygame_viz.py visualization/gridworld_viz.py	Pygame dual viewer, Matplotlib comparison charts
Utilities	utils/registry.py	Algorithm registry, map file discovery
Entry Point	main.py	Launcher UI — map/algorithm/speed selection

3.2 Map Format & Terrain Encoding

Maps are stored as plain-text .txt files in assets/maps/. Each character encodes a tile type. The terrain cost model assigns movement costs to reflect real-world traversal difficulty: Floor and Road tiles cost 1, Mud costs

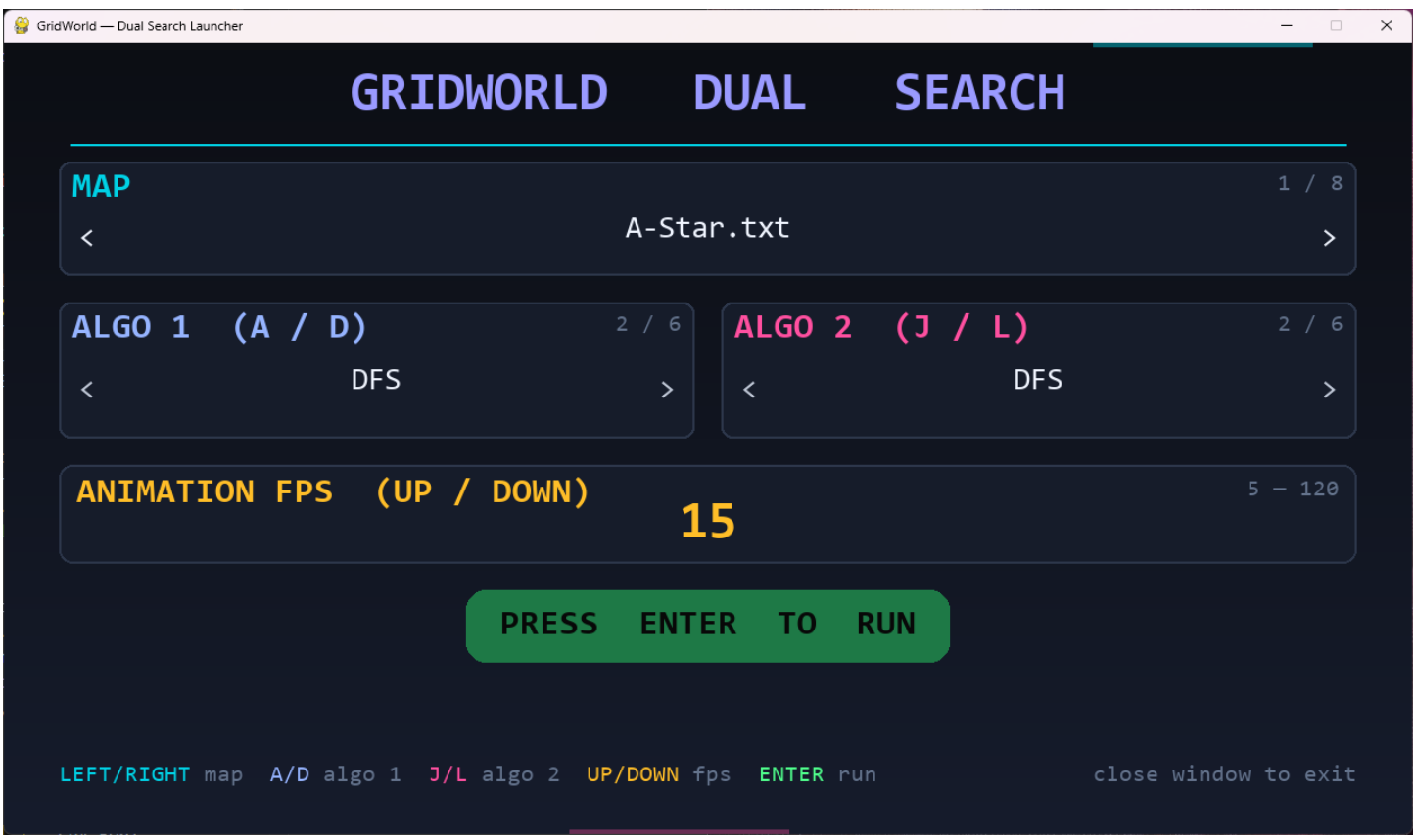
5, and Water costs 10. Walls (O) are impassable. This weighted model allows UCS and A* to demonstrate cost-aware routing behavior.

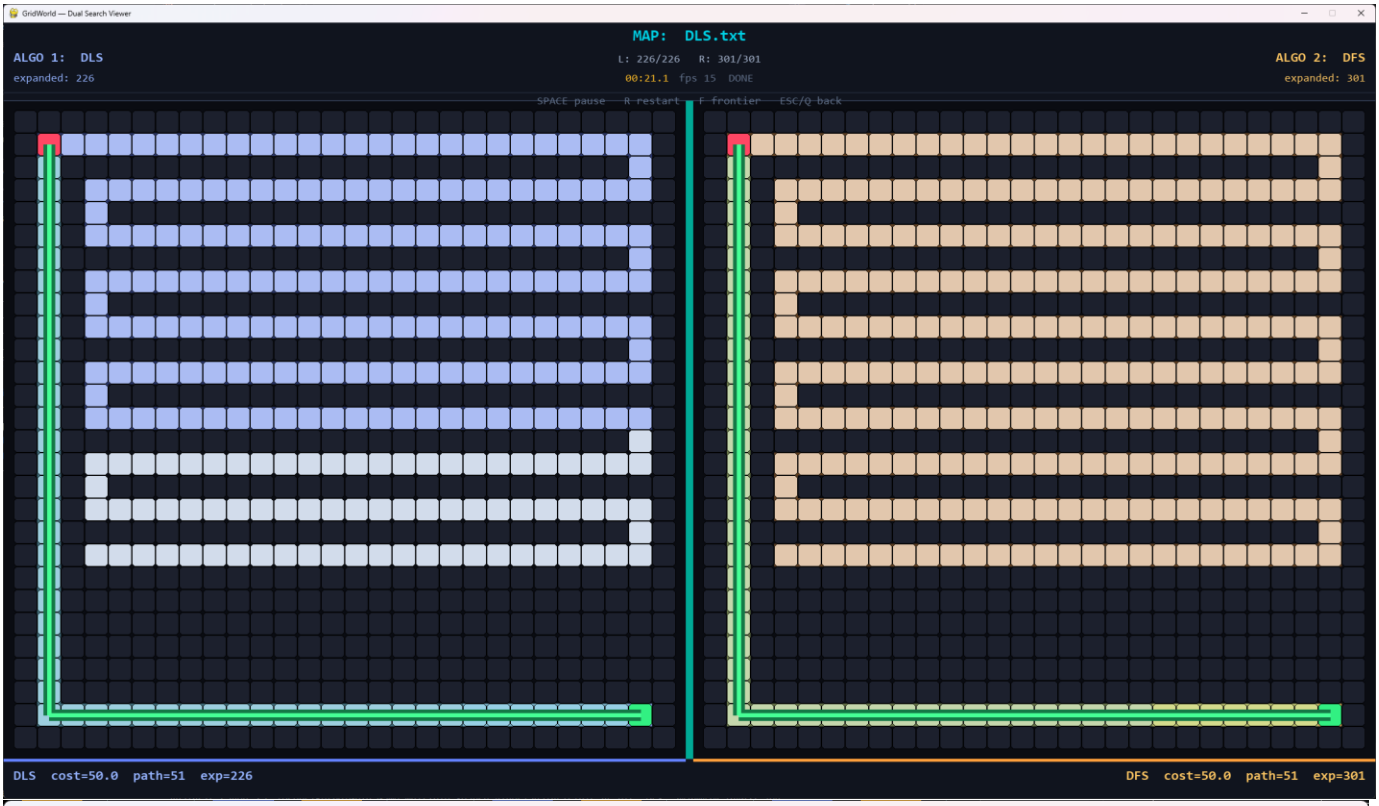
Tile	Char	Cost	Description
Wall	O	—	Impassable; blocks all movement
Floor / Start / Goal	F / S / G	1	Standard traversal cost
Road	R	1	Fast terrain, same cost as floor
Mud	M	5	Slow terrain; discourages routing through
Water	W	10	Very slow; heavily penalized in UCS/A*

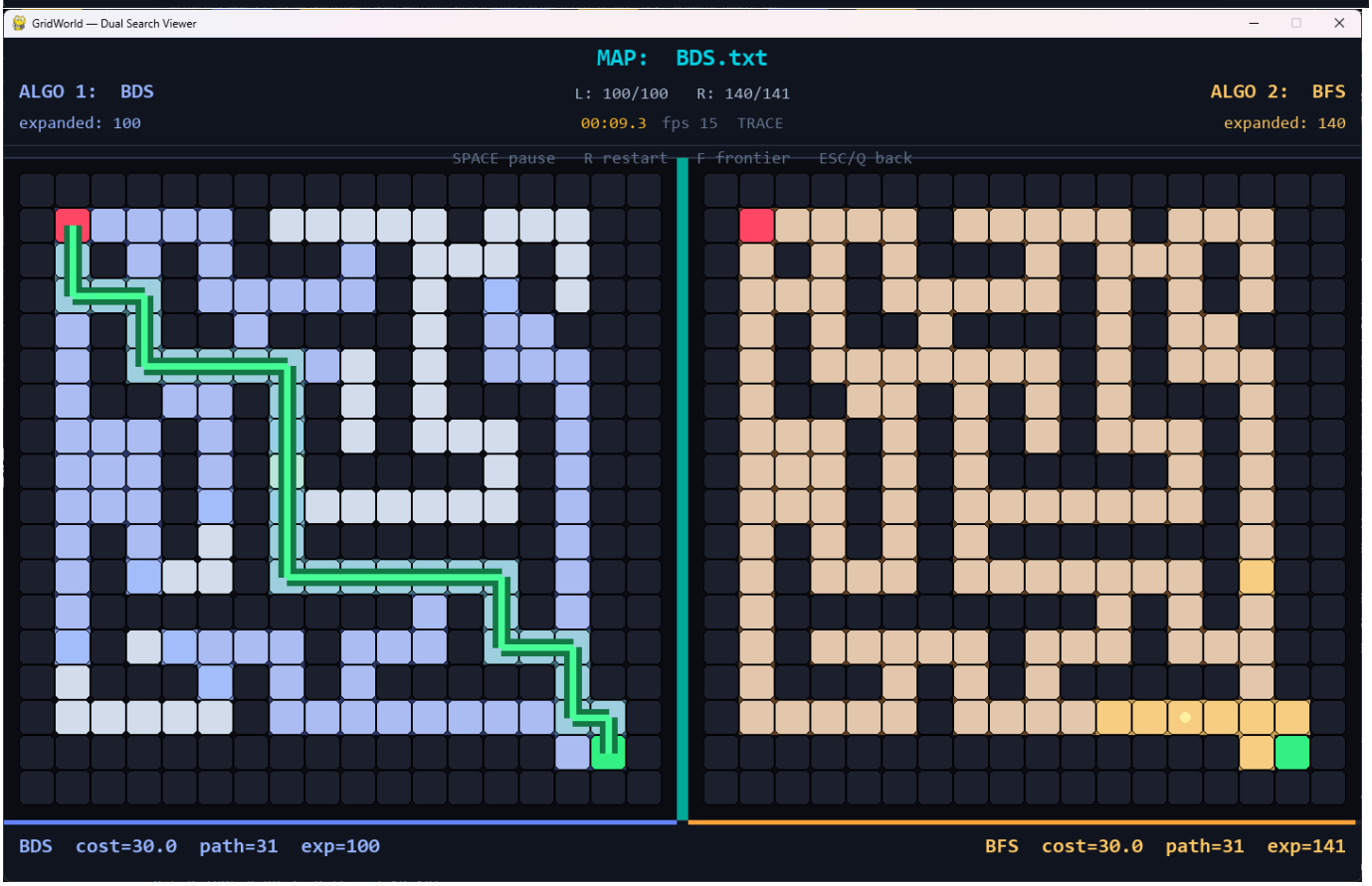
3.3 Algorithm Abstractions

Each algorithm is encapsulated in its own module and yields intermediate states (expanded nodes, current frontier) during execution, enabling the Pygame visualizer to render search progress frame-by-frame. The SearchResult dataclass captures five metrics per run: found (bool), path (list of states), cost (float), expanded (int), and maximum_nodes_waiting_to_explore (int — peak frontier size).

Here are three Screenshots of how the tool looks like (Menu and While exploring):




GridWorld — Dual Search Viewer
MAP: DLS.txt
ALGO 1: DLS
expanded: 226
L: 226/226 R: 301/301
00:21.1 fps 15 DONE
ALGO 2: DFS
expanded: 301
SPACE pause R restart F frontier ESC/Q back
DLS cost=50.0 path=51 exp=226
DFS cost=50.0 path=51 exp=301


GridWorld — Dual Search Viewer
MAP: BDS.txt
ALGO 1: BDS
expanded: 100
L: 100/100 R: 140/141
00:09.3 fps 15 TRACE
ALGO 2: BFS
expanded: 140
SPACE pause R restart F frontier ESC/Q back
BDS cost=30.0 path=31 exp=100
BFS cost=30.0 path=31 exp=141

4. Implemented Algorithms

4.1 Comparison Table

#	Algorithm	Strategy	Data Structure	Optimal?	Notes
1	A*	Best-first, f = g + h	Priority queue	Yes	Manhattan dist. heuristic; requires admissible h
2	BFS	FIFO layer-by-layer	Queue (deque)	Yes*	Optimal when all step costs are equal
3	DFS	LIFO depth-first	Stack (list)	No	Memory-efficient; may find long winding paths
4	DLS	Depth-limited DFS	Stack + depth cap	No	Configurable depth limit; avoids infinite branches
5	UCS	Expand min-cost node	Priority queue	Yes	Handles weighted terrain; generalizes BFS
6	BDS	Dual frontier	Two queues	Yes*	Bidirectional from start + goal; ~halves expansions

* Optimal on uniform-cost maps.

4.2 Key Algorithmic Distinctions

- Informed vs. Uninformed: A* uses the Manhattan distance heuristic to steer toward the goal; all other algorithms explore without goal-proximity knowledge.
- Cost-blind vs. Cost-aware: BFS, DFS, DLS, and BDS treat each step as cost 1. UCS and A* incorporate terrain weights, routing around expensive tiles.
- Data structure drives behavior: BFS explores breadth-first (FIFO), DFS dives deep (LIFO), and UCS/A* always expands the most promising node (min-heap).
- Optimality trade-offs: BFS and BDS guarantee shortest paths on uniform maps; UCS and A* guarantee lowest-cost paths on weighted maps; DFS and DLS offer no optimality guarantee.

5. Experimental Setup

Five custom maps were designed to isolate specific algorithmic trade-offs. Each map was constructed as a rectangular wall-bordered grid with exactly one start (S) and one goal (G). Experiments were run by selecting algorithm pairs via the launcher UI; results were automatically logged and exported as Matplotlib bar charts (saved to assets/analysis/).

Note: Presently there are 6 maps

Map File	Design Intent	Algorithm Pair Tested
A-Star.txt	Open grid to show A* heuristic steering vs BFS	A* vs BFS
BFS_Alternate.txt	Alternating corridor structure	DFS vs BFS

Map File	Design Intent	Algorithm Pair Tested
UCS_Weighted.txt	Mud and Water tiles throughout	UCS vs A*
DLS.txt	Large zigzag dead-end corridor (~250 steps)	DLS vs DFS
BDS.txt	Symmetric open maze	BDS vs BFS

6. Results & Analysis

6.1 A* vs BFS - A-Star.txt

Metric	BFS	A*
Path Cost	30	30
Path Length	31	31
Nodes Expanded	109	65
Max Frontier Size	7	8

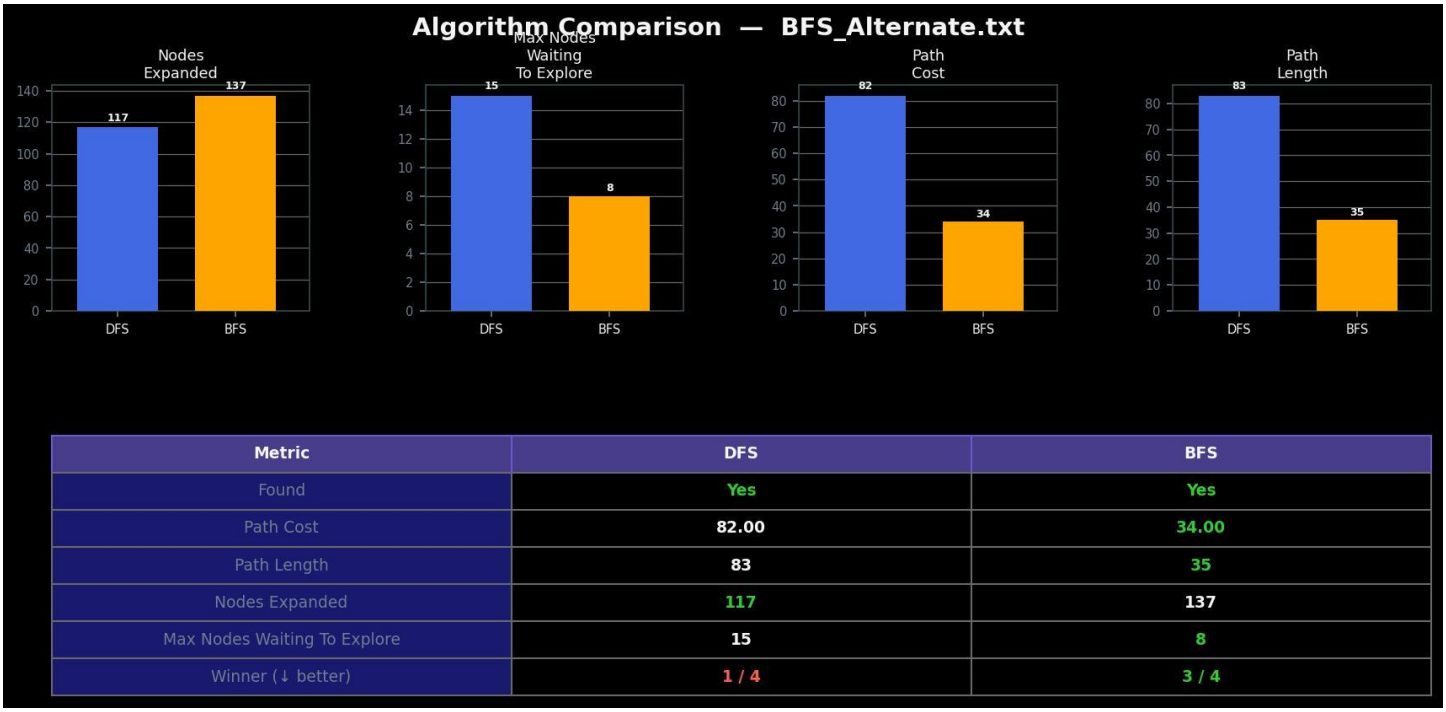
Both algorithms find **the same optimal path** with identical cost and length, as expected on a uniform-cost map where every step costs 1. The key difference is in **nodes expanded**: A* expands to 65 nodes compared to BFS's 109, a ~40% reduction. This is because A*'s Manhattan distance heuristic directs the search toward the goal, pruning branches that lead away from it. BFS, being uninformed, has no choice but to explore every cell at a given distance before moving further. The frontier sizes are comparable (8 vs 7), indicating both algorithms use similar memory.

Metric	ASTAR	BFS
Found	Yes	Yes
Path Cost	30.00	30.00
Path Length	31	31
Nodes Expanded	65	109
Max Nodes Waiting To Explore	8	7
Winner (↓ better)	1 / 4	1 / 4

6.2 DFS vs BFS - BFS_Alternate.txt

Metric	BFS	DFS
Path Cost	34	82
Path Length	35	83
Nodes Expanded	137	117
Max Frontier Size	8	15

This comparison highlights the **non-optimality of DFS**. BFS finds a path of 35 (cost 34), while DFS produces a path more than **twice as long** (83 steps, cost 82). DFS uses a stack and dives deep into the maze's dead ends before backtracking. It happens to find *a* path, but that path winds through the maze inefficiently. BFS, by exploring layer by layer, guarantees it finds the shortest hop-count path. Interestingly, DFS expands fewer nodes (117 vs 137) it stumbled onto a path before exhausting the entire maze - but that "efficiency" came at the cost of path quality.



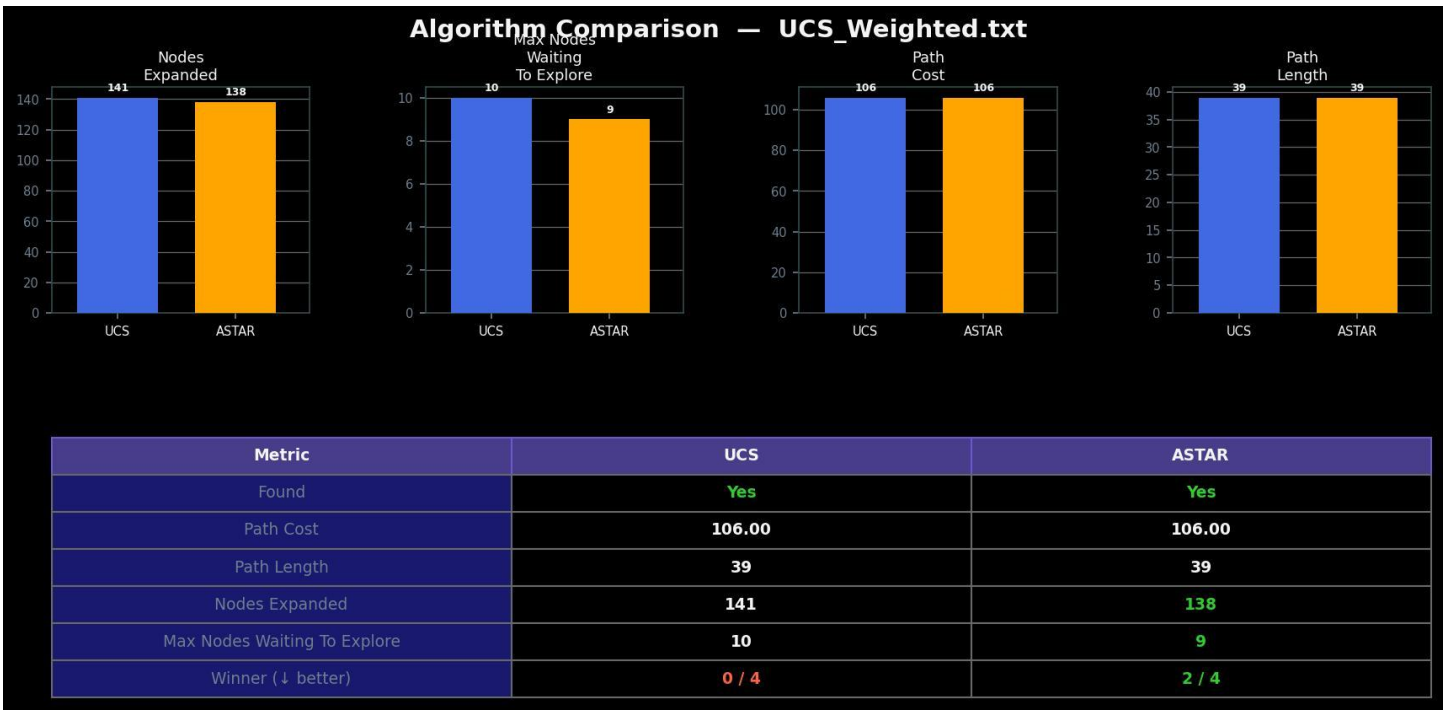
Metric	DFS	BFS
Found	Yes	Yes
Path Cost	82.00	34.00
Path Length	83	35
Nodes Expanded	117	137
Max Nodes Waiting To Explore	15	8
Winner (↓ better)	1 / 4	3 / 4

6.3 UCS vs A* UCS_Weighted.txt

Metric	UCS	A*
Path Cost	106	106
Path Length	39	39
Nodes Expanded	141	138

Metric	UCS	A*
Max Frontier Size	10	9

On a map with **Mud (cost 5)** and **Water (cost 10)** tiles, both UCS and A* now find the **same optimal weighted path** (cost = 106, length = 39). After updating A* to account for actual terrain costs in its path cost calculation, it no longer treats all steps as equal - it properly incorporates edge weights into `g(n)` while still using the Manhattan heuristic for `h(n)`. A* expands to **138 nodes** versus UCS's **141** - a modest ~2% reduction - because the heuristic provides only a slight advantage on this weighted map. Both algorithms use comparable memory (frontier sizes of 9 vs 10). This matchup confirms that a cost-aware A* with an admissible heuristic finds the same cheapest path as UCS, while expanding marginally fewer nodes thanks to heuristic guidance.

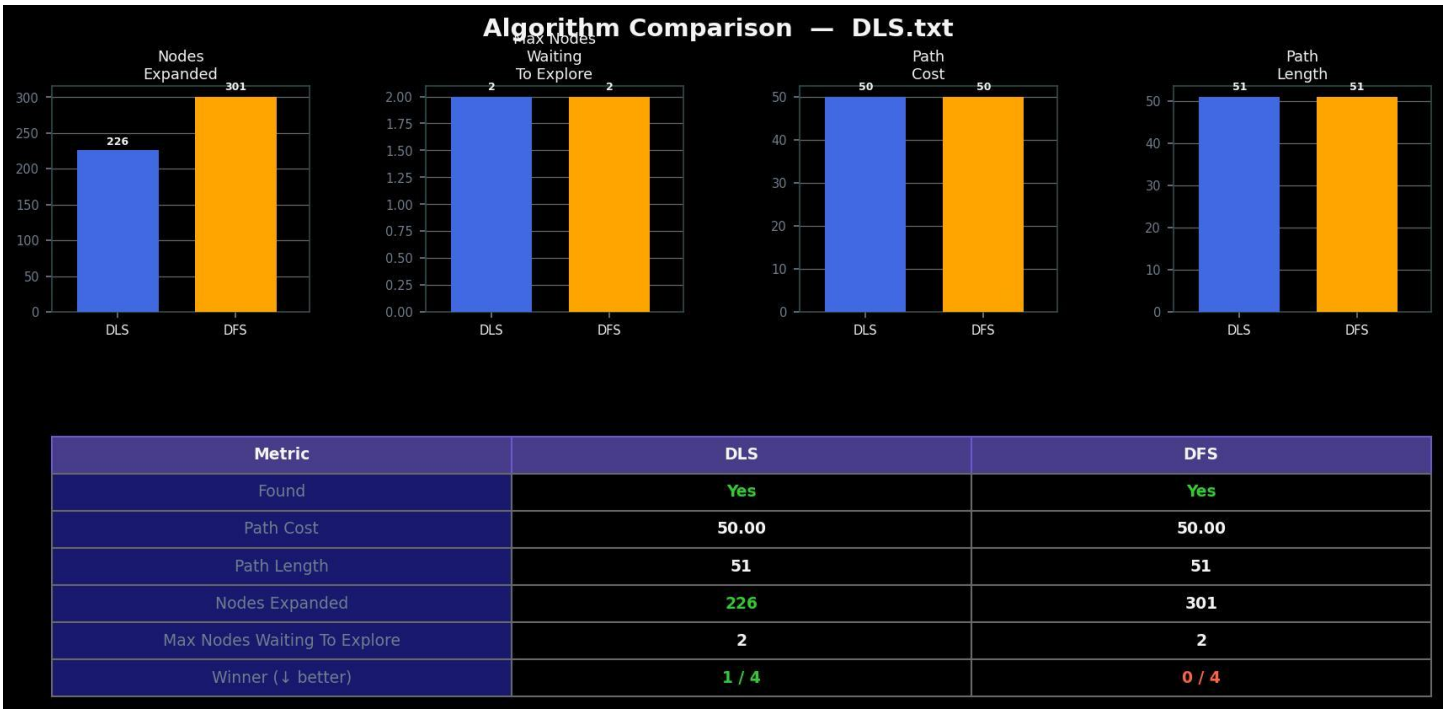


6.4 DLS vs DFS - DLS.txt

Metric	DLS	DFS
Path Cost	50	50
Path Length	51	51
Nodes Expanded	226	301
Max Frontier Size	2	2

Both algorithms find the same path (cost = 50, length = 51), but DLS expands 25% fewer nodes (226 vs 301). The map was designed with a large zigzag dead-end corridor approximately 250 steps deep. DFS, following its LIFO strategy, enters this dead end first and exhausts it before backtracking. DLS,

with a depth limit of 175, cuts off exploration midway through the dead end, saving 75 expansions. The identical frontier size (2) reflects that both algorithms maintain a very narrow active frontier at any point — the branching factor in this corridor is low.

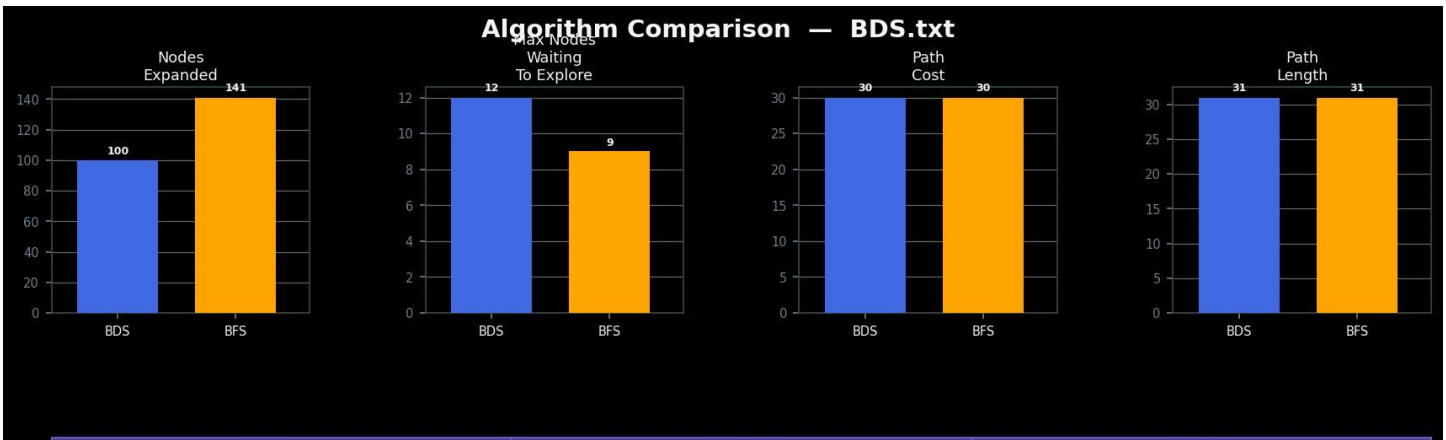


Metric	DLS	DFS
Found	Yes	Yes
Path Cost	50.00	50.00
Path Length	51	51
Nodes Expanded	226	301
Max Nodes Waiting To Explore	2	2
Winner (↓ better)	1 / 4	0 / 4

6.5 BDS vs BFS - BDS.txt

Metric	BDS	BFS
Path Cost	30	30
Path Length	31	31
Nodes Expanded	100	141
Max Frontier Size	12	9

BDS and BFS find identical optimal paths (cost = 30, length = 31), confirming that BDS preserves BFS's optimality on uniform maps. BDS expands 29% fewer nodes (100 vs 141) by searching from both start and goal simultaneously, effectively halving the radius of each explored region. The trade-off is a larger peak frontier (12 vs 9), since BDS maintains two separate queues. The savings are consistent with the theoretical prediction that bidirectional BFS reduces explored volume from O(b^d) to approximately O(b^(d/2)).



Metric	BDS	BFS
Found	Yes	Yes
Path Cost	30.00	30.00
Path Length	31	31
Nodes Expanded	100	141
Max Nodes Waiting To Explore	12	9
Winner (↓ better)	1 / 4	1 / 4

7. Discussion

7.1 What the Results Show Collectively

Across all five experiments, the results confirm the theoretical properties of each algorithm. A key takeaway is that informed Ness (A*) provides the largest single reduction in node expansions, while bidirectionality (BDS) provides the most consistent improvement within the uninformed family. The DLS result demonstrates that even simple modifications to DFS can yield meaningful savings when map topology is known in advance.

7.2 Limitations.

- DLS performance is sensitive to the choice of depth limit; an automated iterative deepening strategy (IDA*) would remove this manual parameter.
- Some of the Maps were hand-designed (DLS.txt) to highlight specific trade-offs; results on randomly generated maps may differ.

7.3 Future Work

- Implement IDA* and Jump Point Search for comparison.
- Add support for dynamic obstacles and online replanning (D* Lite).
- Extend terrain cost model with diagonal movement.
- Export results to CSV for statistical analysis across many random maps.

8. Conclusion

This project implemented and experimentally evaluated six classical search algorithms within a unified interactive visualization framework. The results empirically confirm the theoretical trade-offs documented in the AI literature: A*'s heuristic reduces node expansions dramatically on uniform maps; UCS correctly minimizes terrain cost on weighted maps; BDS achieves consistent ~29% node reduction over BFS; and DLS demonstrates targeted savings over DFS when dead-end corridors are present.

The GridWorld Dual Search Visualizer serves as both a practical demonstration of algorithm behavior and an extensible platform for further experimentation. All source code, map files, and analysis outputs are available at github.com/SwapnanilBala.

References

Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. Numerische Mathematik, 1, 269–271.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

Pohl, I. (1971). Bi-directional Search. Machine Intelligence, 6, 127–140.

Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Python Software Foundation. (2025). collections — Container datatypes. Python 3.12 Documentation. https://docs.python.org/3/library/collections.html

Python Software Foundation. (2025). heapq — Heap queue algorithm. Python 3.12 Documentation. https://docs.python.org/3/library/heapq.html

Python Software Foundation. (2025). dataclasses — Data Classes. Python 3.12 Documentation. https://docs.python.org/3/library/dataclasses.html